

MASt3R-based Gaussian Splatting Pipeline Technical Specification

1.0 Overview

This document presents the technical specification for a 3D reconstruction pipeline designed to solve **two major challenges in modern 3D reconstruction: the complexity of traditional Structure from Motion (SfM) and the performance limitations of typical hardware**. **The pipeline aims to democratize the process of generating high-fidelity 3D scenes from 2D image collections**. At its core is the strategic decision to replace traditional SfM tools with the deep learning model MASt3R, **which** significantly streamlines the initial reconstruction process and enables an end-to-end automated workflow. This specification details the pipeline's architecture, the technical details of each component, and the key technical decisions that underpinned its design. This allows developers and operators to accurately understand the overall system and how it works. The following sections detail the infrastructure requirements essential for running the pipeline.

1.1 Purpose and characteristics of the pipeline

The main goal of this pipeline is to automate the process of generating high-quality 3D scenes from multiple 2D images. To achieve this, it has the following key features:

- **Replacement of the SfM process with MASt3R:** The SfM tools, such as COLMAP, which were previously used for feature point matching and camera pose estimation, are replaced with a single deep learning model, MASt3R. This simplifies the initial 3D reconstruction process and improves the efficiency of the entire workflow.

- **DINO-based intelligent pair selection:** Using the DINOv2 model, global features of each image are extracted and the similarity between images is calculated. This allows for intelligent selection of image pairs that are most likely to contribute to 3D reconstruction, reducing unnecessary computational overhead.

- **Biplet-Square Normalization:** A unique dual-crop strategy is employed to generate two overlapping square crops (left and right for landscape and top and bottom for portrait) depending on the image orientation (portrait/landscape). This standardizes image size for subsequent processing while maximizing the preservation of peripheral information within the image.

- **Strict memory management:** Memory efficiency is thoroughly pursued using multiple techniques such as image resizing, limiting the number of pairs, and downsampling of point clouds. This enables stable operation even in environments with limited hardware resources such as GPU memory.

- **End-to-end automation:** It functions as a comprehensive framework that covers all processes, from installing dependencies, to data preprocessing, 3D reconstruction, and finally training the Gaussian Splatting model.

1.2 Overall Architecture

This pipeline has a linear data flow architecture consisting of the following consecutive stages:

Environment construction:

- **Input:** None

Processing:

Basic dependencies of the system, MASt3R, and Gaussian Splatting repository and install and configure the submodules.

- **Output:** Executable pipeline environment

Image preprocessing:

- **Input:** Original 2D images

- **Processing:** Apply Biplet-Square Normalization to each image to generate a standardized square crop image.
- **Output:**

Processed images • Image pair

selection: • **Input:**

Processed images • **Processing:**

Extract global features from each image using the DINOv2 model and perform similarity matching.

- **Output:** A list of image pairs to

be used for reconstruction • 3D reconstruction

with MAST3R: • **Input:** Processed

images and selected image pairs • **Processing:** The

MASt3R model takes the image pairs as input and performs inference. • **Output:** A

scene object containing the globally aligned point cloud (pts3d) and the camera pose

(camera-to-world) • **Conversion to COLMAP format:**

- **Input:** The scene object

output by MAST3R • **Processing:** The output of

MASt3R is converted to the COLMAP binary format. This process involves:

This includes pose inversion (world-to-camera transformation) and scaling of camera intrinsic parameters.

- **Output:** Three files: cameras.bin, images.bin, points3D.bin • **Gaussian**

Splatting Model Training: • **Input:**

COLMAP format data and processed images •

Processing: Train the Gaussian Splatting model based on the COLMAP data . • **Output:**

A renderable 3D scene model (point_cloud.ply)

2.0 environment construction

The environment construction phase is crucial to ensure stable execution of the pipeline. During this initial stage , all dependencies are correctly installed and configured, from basic Python libraries to specialized frameworks such as MAST3R and Gaussian Splatting, and even their submodules. Building this foundation correctly ensures that subsequent complex computations run smoothly.

2.1 Basic Dependencies

The set of libraries that form the foundation of the pipeline are installed by the setup_base_environment function . **The main packages are as follows:** •

torch, torchvision, torchaudio: **Computational infrastructure**

for deep learning • opencv-python,

pillow: **Image processing** • transformers:

Use of DINOv2 model • pycolmap: **For handling COLMAP**

format data structures • numpy==1.26.4: **Fixed to a specific version to**

avoid compatibility issues • scipy, psutil, tqdm: **Utilities for scientific calculations, memory monitoring, progress b**

2.2 MAST3R Setup The

MASt3R framework, which is the core of 3D reconstruction, is set up by the setup_mast3r function in the following steps:

1. Clone the MAST3R repository from GitHub. 2. Install the

dependent submodules dust3r and croco . 3. Install the dependent packages listed

in requirements.txt .

4. Download the pre-trained model

`weights (MASt3R_ViTLarge_BaseDecoder_512_catmlpdpt_metric.pth)` .

5. Import the AsymmetricMASt3R model to confirm the installation was successful.

Verify the following:

2.3 Gaussian Splatting Setup The Gaussian

Splatting framework that generates the final 3D scene is set up by the `setup_gaussian_splatting` function as follows:

1. **Clone the** Gaussian Splatting repository from GitHub .
2. **Install the Python dependencies.** 3. **Install the two submodules** `diff-gaussian-rasterization` and `diff-gaussian-rasterization` , which are essential for the pipeline's performance. **Build and install** `simple-knn` .

2.4 Memory Management

Utilities This pipeline is equipped with the following memory management utilities to ensure stable execution of computationally intensive processes.

- **`clear_memory()`:** Called after the completion of each major step, it frees the GPU cache using `torch.cuda.empty_cache()` and forces a garbage collection of CPU memory using `gc.collect()`, thereby maximizing the memory available for the next processing stage.
- **`get_memory_info()`:** At any point during pipeline execution, this function logs the current GPU and CPU memory usage, making it easier to monitor resource consumption and identify bottlenecks.

3.0 Pipeline Components and Data Flow

This section provides a step-by-step technical description of each element of the pipeline, following the data flow. Starting from image normalization to the final 3D model training, we clarify the architectural purpose, logic, inputs, and outputs of each component , providing a detailed understanding of how the entire system works.

3.1 Step 1: Image Preprocessing (Bi-plet-Square Normalization) This

normalization stage is essential for stabilizing the performance of the subsequent model. The `normalize_image_sizes_bi-plet` and `generate_two_crops` functions employ a bi-plet-square strategy to maximize the preservation of peripheral visual information compared to simple center cropping, thereby providing a richer context for feature point matching. The process is as follows:

Orientation determination: Compare the width and height of each image to determine whether it is landscape or portrait.

Masu.

Dual Cropping: Generates two overlapping square crops based on the determined orientation.

Completed.

- **For landscape images (width W x height H):** A square with height H on one side is cut out at two points , from the coordinates (0, 0) and

(WH, 0). • **For portrait images:** A square with width W on one side is cut out at two points, from the coordinates (0, 0) and (0, WH). **Cut it out.**

- **Resize:** Each generated cropped image is resized to the specified resolution (e.g. size =1024) using the high-quality downsampling algorithm `Image.Resampling.LANCZOS` .

3.2 Step 2: Image Pair Selection (DINO-based Pair Selection) The

get_image_pairs_dino function intelligently selects the most relevant image pairs to maximize the computational efficiency of 3D reconstruction. The mechanism is as follows:

1. **Feature extraction:** Using the pre-trained model DINOv2 (facebook/dinov2-base) , we extract a global feature vector from each normalized image. This vector summarizes the visual content of the entire image.
2. **Similarity calculation:** Using the extracted feature vectors of all images, we create a brute-force similarity matrix ($\text{sim} = \mathbf{g} @ \mathbf{g}^T$) .
3. **Top-K Pair Construction:** For each image, we identify the top GLOBAL_TOPK (value: 20) most similar images based on the similarity matrix to form initial candidate pairs.
4. **Diverse Pair Selection:** If the total number of candidate pairs exceeds a defined upper limit (max_pairs) , the select_diverse_pairs strategy is applied. This algorithm calculates the number of times each image appears in a pair (image_counts) and calculates a score (pair_score) that prioritizes pairs containing images with fewer appearances. This prevents the formation of visual "hubs" (where a few images dominate a pair) and ensures more spatially robust and complete reconstructions, especially in datasets with linear motion.

3.3 Step 3: 3D Reconstruction (MASt3R Reconstruction) The

run_mast3r_pairs function performs the core 3D reconstruction process of the pipeline.

1. **Loading the model and data:** The pre-trained Asymmetric MASt3R model is loaded into memory , while the input images are prepared by resizing them to a lower resolution (MASt3R_IMAGE_SIZE = 224) to reduce memory consumption.
2. **Inference:** The selected image pairs are passed to the model's inference function, which calculates the relative color between each pair. The melapose and depth information are predicted.
3. **Global alignment:** Pass the output of the inference function to the global_aligner function (We process the data using the GlobalAlignerMode.PointCloudOptimizer mode (GlobalAlignerMode.PointCloudOptimizer mode) to integrate the information from all pairs and construct a globally consistent 3D scene. The number of optimization iterations is set to niter=150 to balance accuracy and processing speed.

3.4 Step 4: Data Conversion to COLMAP Format

This data conversion stage plays a crucial role in meticulously converting MASt3R's native scene representation into the strict, standardized COLMAP binary format required by Gaussian Splatting .

1. **Data extraction:** Extract 3D point coordinates (pts3d), color information, camera pose, and focal length from the MASt3R scene object .
2. **Point Cloud Downsampling:** If the total number of 3D points exceeds the specified limit (max_points) , the number of points is reduced by random sampling. This is an important safety mechanism to prevent memory overflow in subsequent processing.
3. **Pose Conversion:**

The most important process is to convert the camera-to-world (c2w) format camera pose output by MASt3R into the world-to-camera (w2c) format required by COLMAP by calculating the inverse matrix using `np.linalg.inv` . Furthermore, the 3x3 rotation matrix is converted into a 4-element quaternion (w, x, y, z) required by the images.bin format using the `rotmat2qvec` function.

4. **Scaling camera internal parameters:** Scale the calculated focal lengths (focals) and principal points (pp) based on the MASt3R's internal processing size (224x224) to match the dimensions of the original processed image.

5. **Serialization in binary format:** All processed data (camera, images, 3D points) is serialized in **It writes directly** to three COLMAP standard binary files: cameras.bin, images.bin, and points3D.bin .

3.5 Step 5: Gaussian Splatting

The final stage of the model training pipeline, the `train_gaussian_splatting` function, uses the generated COLMAP **data** to complete the 3D scene. Input : **The input for this stage**

is the binary COLMAP file generated in the previous step .

directory and a directory containing the processed images.

- **Execution of training:** Run the `train.py` script in the Gaussian Splatting repository using the subprocess code. **This is done via the**

- **Performance Tuning Parameters:** The following main command line arguments are used to manage performance and memory usage during training: • `--resolution 2:`

Reduces image resolution by a factor of 2 to reduce memory load. • `--`

`densify_grad_threshold 0.001:` **Sets a higher density threshold to reduce the number of Gaussians created.**

- `--densification_interval 200:` **Reduces the frequency of Gaussian densification, reducing the amount of computation.**

(densifying less frequently to lower the computational load) • Output: When training is complete, the final deliverable, a trained point cloud model (`point_cloud.ply`), **is generated in the specified output directory .**

4.0 Configuration Parameters

The pipeline has the flexibility to adapt to different datasets, hardware constraints, and desired output quality through configuration parameters, which provide a key interface for precisely controlling the pipeline's behavior.

4.1 Global Configuration

These parameters defined in the `Config` class specify the basic behavior of the entire pipeline. | Parameter | Value | Description || ----- | ----- | ----- || `N_KEYPOINTS` | 8192 | **Maximum number of feature points to extract.** || `IMAGE_SIZE` | 1024 | Image size after Biplet-Square Normalization. || `GLOBAL_TOPK` | 20 | **Maximum number of similar pairs to select for each image in DINO pair selection.** || `DINO_MODEL` | "facebook/dinov2-base" | Path to the **DINO model used for feature extraction .** || `MASt3R_MODEL` | .../MASt3R_ViT...pth | Path to the pre-trained MASt3R model . || `MASt3R_IMAGE_SIZE` | 224 | **Image size used during MASt3R inference. Set to a small size to save memory.** || `DEVICE` | 'cuda' or 'cpu' | **Device to use for computation.** |

4.2 Pipeline Runtime Parameters

These parameters, passed as arguments to the `main_pipeline` function, are used to tune the behavior of the pipeline for each individual run. | Parameter | Example Default Value | Description || ----- | ----- | ----- || `square_size` | 1024 | **Target square size for image preprocessing.** || `iterations` | 1000 | **Number of training iterations for Gaussian Splatting.** || `max_images` | 30 | **Maximum number of original images to process.** || `max_pairs` | 1000 | **Maximum number of image pairs to use for reconstruction with MASt3R.** || `max_points` | 1000000 | **Maximum number of 3D points to convert to COLMAP format.** |

5.0 Key Technical Decisions

The effectiveness of this pipeline is based not only on the individual processing steps, but also on several key architectural decisions made to improve speed, resource management, and final reconstruction quality.

This section analyzes the rationale behind these crucial choices.

5.1 MAST3R as a replacement for COLMAP

The strategic decision to replace the traditional SfM pipeline with MAST3R fundamentally transforms the initial reconstruction phase. This decision combines the fragile, multi-stage process of feature extraction, matching, geometric validation, and incremental reconstruction into a single, end-to-end differentiable model. The result is a more robust and predictable workflow with significantly fewer points of failure and reduced configuration complexity.

5.2 The Role of Biplet-Square Normalization

The use of biplet-square normalization, a dual cropping method, instead of a simple center crop plays an important role in improving the robustness of the reconstruction. This method intentionally preserves visual information that is often lost from the periphery of the original image. The preserved data provides more context for feature point matching, which can lead to more accurate and stable camera pose estimation, especially when there is little overlap between images.

5.3 Strict Memory

Management Strategy At the core of the design philosophy of this pipeline is a "resource-aware design philosophy." This is not a single feature, but a consistent strategy for ensuring executableness with limited hardware resources. This strategy operates at multiple levels of abstraction.

- **Inference level:** In the most computationally intensive inference step of MAST3R, we dramatically reduce the GPU memory footprint by reducing the input image resolution to `MAST3R_IMAGE_SIZE=224`.
- **Data structure level:** Limiting the maximum number of point clouds with the `max_points` parameter acts as a crucial safeguard to prevent memory exhaustion during the data conversion and Gaussian Splatting training stages.
- **Process level:** The `max_pairs` parameter limits the number of pairs that MAST3R processes, directly controlling the scope of the reconstruction and resource consumption. Furthermore, explicit garbage collection with `clear_memory()` ensures resource release between key stages, ensuring stability of the entire pipeline. These architectural decisions combine to make the pipeline not just functional, but also robust, efficient, and adaptable to real-world constraints (e.g., limited GPU memory). The sum of these strategic choices creates a reliable pipeline capable of tackling modern 3D reconstruction tasks.